共通演習：準備・接続仕様・統合

第13グループ / 夜の学校から脱出 / 2026年11月3日

0. この教材の使い方

これはグループ用の自主教材です。教員配布資料そのものではありません。代表者を含む5人が、学校脱出の共通ベースに新しい機能を追加します。

対象は代表者リポジトリのコミット 33fe9905c183b321d1988d66041feb8b7e74998f。教材中のファイル名・クラス名はこの版に合わせています。行番号より、クラス名とメソッド名で検索してください。

最初に全員がこの資料を読み、次に担当の01〜05を読みます。01〜05の仕様は統合できるよう揃えた提案です。全員で採用を確認し、変更するときはこの共通仕様も更新します。

試作品にある移動・ダッシュ・鍵・巡回・単純追跡・HP・結果画面は共通ベースです。新しい個人実装と区別し、READMEに出典と担当を記録してください。演習のTODOには自分で処理を書き、説明できるようにします。

1. 全員のゴールと担当

担当	新しく実装するもの	主な提出ファイル
代表者 kusupon	2ステージ、World、配置検証、統合	world.py / stages.py
c0a25107a3	捜索状態、巡回復帰、足音への反応	enemy_ai.py
メンバーA	隠れる、出る、足音の発生	player.py
メンバーB	色付き鍵、扉、スイッチ	gimmicks.py
メンバーC	ミッション、得点内訳、記録保存	missions.py

氏名が未定の3人は相談で割り当てます。1人が全機能を実装する課題ではありません。既存機能の移動だけを追加成果に数えず、各資料の必須課題まで実装します。

2. 最初の準備

ブラウザで自分の授業用アカウントにログインし、代表者の kusupon/ProjExD_Group13 をForkします。フォーク先を確認してから自分のPCへクローンします。

以下の「自分のID」「学籍番号」は各自の値に置き換えます。既存のex5がある場合は別名で保管し、その上へクローンしないでください。

```
git clone https://github.com/自分のID/ProjExD_Group13.git ex5
cd ex5
git remote add upstream https://github.com/kusupon/ProjExD_Group13.git
git switch -c 学籍番号/機能名
py -3.13 -m venv .venv
.¥.venv¥Scripts¥python.exe -m pip install -r requirements.txt
.¥.venv¥Scripts¥python.exe game.py
```

c0a25107a3のPCは準備済みです。lec05/第5回.code-workspaceを開き、F5で実行できます。ブランチはC0A25107/enemy-ai。origin未設定なので、フォークと認証を整えてからpushします。

合格確認：タイトルが出る、Enterで開始、矢印で移動、Shiftでダッシュ、Escで一時停止、×で終了できる。

3. 現在のコードを読む

現在の名前	処理	演習後の担当
make_map / GRID / WALLS	固定マップと壁	Worldへ集約
center / cell_at / find_path / visible	座標・経路・視線	Worldへ移す
Player	移動・HP・スタミナ	主人公担当が継承して拡張
Guard	巡回・単純追跡	敵AI担当が拡張
Game.collect_items	鍵・回復	鍵は仕掛け担当、回復はGameに維持
Game.update_guards	接触と被弾	Gameが窓口になる
Game.check_result	クリア・失敗	判定後にミッションへ通知

現在の Guard.update(dt, player) のplayerはVector2です。演習後はPlayerオブジェクトを渡します。引数の変更は呼び出し元も同時に直してください。

4. 共通データの約束

時間は秒、画面座標はピクセル、マスは (列, 行)。左上が原点です。grid[row][col] の順番と座標の (col, row) を混同しないでください。

共通の型は添付の starter/contracts.py にあります。演習開始時に代表者がゲームのルートへ配置します。Python 3.10以上で使える型注釈を採用しています。コード例のpgは import pygame as pg の略称です。抜粋には必要なimportを省略しているので、新しいファイルの先頭へ追加します。

```
Cell = tuple[int, int]

@dataclass(frozen=True)
class NoiseEvent:
    position: tuple[float, float]  # 発生時の座標のコピー
    radius: float                  # 聞こえる距離
```

足音はそのフレームの全敵に同じtupleを渡します。敵ごとにpopしないでください。全敵の更新後に破棄し、次フレームへ持ち越しません。時刻の保存は不要です。

Stageには、壁、開始地点、巡回経路、鍵、出口、回復、隠れ場所、扉、スイッチを持たせます。鍵のIDは red / blue / green のような文字列で統一します。鍵の種類と取得数を別々の名前で表さないようにします。

5. Worldの共通API

名前	返すもの・意味
world.stage	現在のStage
world.walls	壁と未解錠の扉のRect一覧
world.center(cell)	マス中心のVector2
world.cell_at(position)	画面座標に対応するCell
world.walkable(cell)	現在通れるか。範囲外はFalse
world.find_path(start, goal)	開始マスを除く経路。到達不能なら空リスト
world.visible(a, b)	壁・閉じた扉に遮られなければTrue
world.open_door(door_id)	初回の開放ならTrue、既に開いていればFalse

Worldは1プレイに1個作ります。扉を開いたら、壁・経路・視線のすべてに反映します。今回の必須課題では扉は開く一方とし、途中で閉じません。

経路が空の場合は「開始と目的地が同じ」か「到達不能」です。先に start == goal を確認して区別します。

6. 4機能の共通API

クラス	統合時の呼び出し
Player	Player(stage.player_spawn, world)
Player	handle_event(event, world) / update(dt, keys, world)
Guard	Guard(route, world) / update(dt, player, world, noises)
Gimmicks	Gimmicks(world) / handle_event(event, player) / update(player)
MissionTracker	MissionTracker() / on_key(key_id) / on_damage() / finish(...)

Playerは position: Vector2、rect: Rect、hidden: bool、hp・staminaを公開します。updateはそのフレームの tuple[NoiseEvent, ...] を返します。take_damage、recover、drawは既存の役割を維持します。

Guardは位置とstunnedを持ち、HPを直接変更しません。Gimmicksは inventory: set[str] と can_exit() -> bool を持ちます。鍵の取得済み状態はここだけで管理し、Game.keysとの二重管理をやめます。

全機能の描画は draw(screen) が基本です。ただし既存Guardの draw(screen, font) は、そのまま使って構いません。更新処理をdrawへ書かないでください。

7. 共通準備を分担する

代表者はcontracts.pyとStage/Worldの枠を登録します。各自は担当クラスを別ファイルへ移し、同じ挙動で起動する段階を最初のチェックポイントにします。

循環importを防ぐため、enemy_ai.pyやplayer.pyからgame.pyをimportしません。定数は settings.py、共通型は contracts.py に置き、必要なWorldは引数でもらいます。

この準備コミットの後に個人機能を追加します。相手の完成を待つ間は starter/sandbox.py の小さなWorldと仮Playerでロジックを確認できます。このsandboxはゲーム本体ではなく練習用です。

本人のブランチでは、まず新規のenemy_ai.pyに取り組みます。全員がgame.pyの同じ行を同時に編集することを避け、Gameへの接続は代表者と順番を決めます。

8. Gameへ接続する順序

resetで選択中のStageからWorldを作り、Player、Guard一覧、Gimmicks、MissionTrackerを作り直します。タイトルで選んだstage_indexはresetで0に戻さず保持します。回復の使用状態もresetで戻し、result=None、result_saved=Falseに初期化します。

1フレームの順序は次のとおりです。下の例は接続位置を示す抜粋で、Game全体の置き換えコードではありません。

```
# stateがplayingのときだけ実行する
self.time_left = max(0.0, self.time_left - dt)
noises = self.player.update(dt, keys, self.world)
before = self.gimmicks.inventory.copy()
self.gimmicks.update(self.player)
for key_id in self.gimmicks.inventory - before:
    self.missions.on_key(key_id)
for guard in self.guards:
    guard.update(dt, self.player, self.world, noises)
    if not self.player.hidden:
        hit = self.player.position.distance_to(guard.position) < 25
        if hit and self.player.take_damage():
            guard.stunned = GUARD_STUN_TIME
            self.missions.on_damage()
self.check_result()
```

回復処理はcollect_itemsから残し、world.stage.heal_cellを参照します。hidden中は取得しません。状態の終了優先順位はHP0、時間切れ、出口到達の順です。

playing中のKEYDOWNは、Player.handle_event（Eで隠れる）とGimmicks.handle_event（Fでスイッチ）へ渡します。Escでpausedへ変わったフレームには、それらへ渡しません。

9. 終了と描画の接続

クリア条件は gimmicks.can_exit() と出口への距離です。結果が決まった瞬間にだけMissionTracker.finishを呼びます。draw_overlayから保存しないでください。

```
self.result = self.missions.finish(
    won=self.state == 'won',
    time_left=self.time_left,
    hp=self.player.hp,
    stage_id=self.world.stage.stage_id,
)
```

finishは同じプレイでは同じ結果を返す設計です。保存は self.result_saved がFalseかつwonのときに1回だけ試し、成否にかかわらずTrueにします。失敗したら画面へ「記録を保存できませんでした」と表示してゲームは継続します。

Worldが床・壁・扉を描き、Gimmicksが鍵・スイッチを描き、PlayerとGuardがキャラクターを描きます。HP、所持鍵、残り時間はHUDへ、得点内訳は結果画面へ渡します。

10. 統合と提出

統合順はWorld、仕掛け、主人公、敵AI、ミッションです。1つ取り込むたびに起動、鍵取得、被弾、一時停止、終了を確認します。

作業をコミットしてからmainを同期します。履歴が分岐してff-only更新が止まったら、自分の機能ブランチで内容を確認して git merge upstream/main を行います。競合を解消し、実行してからコミットしてください。

第5回は各自の担当ブランチをpushし、README、コード、ベースとの差分の順でPDFを用意します。第6回にPRで統合します。提出方法と採点は教員の講義資料を優先します。

READMEには、操作、基本機能、本人の追加機能、担当、必要ライブラリ、参考資料、確認方法を記載します。ブランチ名は本人なら C0A25107/enemy-ai、他の人は自分の学籍番号にします。

11. 完成チェック

- 全員のPCで起動でき、担当以外の人が操作説明だけで遊べる。

- 両ステージで鍵と仕掛けに到達でき、脱出できる。

- 敵の視界、隠れる、足音、開いた扉の判定が一致する。

- 一時停止で時間が止まり、再挑戦で全状態が初期化される。

- 保存ファイルが壊れていてもゲームが起動する。

- 個人機能の説明と実際の差分が一致する。

参照

- 基準コード：https://github.com/kusupon/ProjExD_Group13/blob/33fe9905c183b321d1988d66041feb8b7e74998f/game.py

- 教員資料：第5回 p.8～10、p.13、p.21～25（配布PDFを参照。公開教材には同梱しない）

- Pygame Rect : https://www.pygame.org/docs/ref/rect.html

- Pygame Vector2 : https://www.pygame.org/docs/ref/math.html

- Git merge : https://git-scm.com/docs/git-merge